



Practical No. :- 01

Aim: Create an application to demonstrate Node.js modules.

Theory:

- Node.js is an open-source, cross-platform JavaScript runtime environment that runs on Chrome's V8 engine. It allows JavaScript to be executed on the server side, enabling developers to build scalable, fast, and efficient web servers applications using JavaScript.

Why Node.js?

- Node.js uses a single-threaded, non-blocking, asynchronous model, making it efficient and lightweight compared to traditional web servers like PHP or ASP.

Features:

- Single-threaded, event-driven, and non-blocking
- Highly scalable and memory-efficient.

modules in Node.js -

1. Built-in modules -

- Node.js comes with several inbuilt modules such as:
 - http - Create web servers.
 - url - Parse and handle URLs.



- fs - works with file systems
- Path - Handle file paths
- os - Get system information

Example:

```
const http = require('http');  
http.createServer((req, res) => {  
  res.writeHead(200, {'Content-Type': 'text/plain'});  
  res.end('Hello world!');  
}) .listen(3000);
```

2. Local modules

- These are user-created modules that can be reused within your project.

Example: SayHello.js:

```
function sayHello(username) {  
  console.log('Hello $ {username}!');  
}  
  
module.exports = sayHello;
```

3. Third-Party modules -

- These are external packages installed using NPM (Node Package manager).

Install a Package:

```
npm install capitalize
```



Conclusion -

- Node.js revolutionized server-side programming by allowing developers to use JavaScript on the backend.
- It's asynchronous, event-driven, and non-blocking nature makes it ideal for building high-performance, scalable, and real-time applications.
- With the help of modules and NPM packages, Node.js simplifies development and encourages modular, reusable, and maintainable code.
- It continues to be one of the most popular platforms for modern web and API development.



Practical No.: 02

Aim: create an application to demonstrate various Node.js Events.

Theory:

- Node.js is an asynchronous, event-driven JavaScript runtime. It uses an event-driven architecture that performs non-blocking operations efficiently.
- The 'events' module in Node.js allows creating, listening, and handling custom events. It emits named events, and the corresponding functions (listeners or callbacks) are executed whenever the event occurs.
- At the core of this system is the EventEmitter class. All objects that emit events are instances of this class. It provides methods to add, trigger, and remove event listeners.

Example Setup:

Importing EventEmitter:

```
const EventEmitter = require('events');
```

Creating An Instance:

```
const myEmitter = new EventEmitter();
```



Conclusion -

- Node.js's EventEmitter class is a powerful feature that allows applications to handle asynchronous events efficiently.
- It provides a simple way to register, trigger, and remove events, enabling developers to create non-blocking, event-driven applications such as servers, chat systems, and streaming services.



Practical NO. :- 03

Aim: Create an application to demonstrate Node.js Functions.

Theory:

- In Node.js, functions are blocks of reusable code that can be called from different parts of an application. Functions in Node.js are similar to JavaScript functions but often handle asynchronous, non-blocking operations.

Types of Functions in Node.js:

Basic Function:

- A standard function definition as in JavaScript.

Arrow Function:

- A concise way to write functions, often used in callbacks.

Asynchronous Functions:

- Node.js frequently deals with asynchronous tasks such as file operations or HTTP requests. These can be handled using callbacks, Promises, or Async/Await.

callback Functions: Functions passed as arguments to handle results after an async operation.

Promises: Allow cleaner handling of asynchronous code.

Async/Await: Asynchronous code looks synchronous.

Conclusion -

- Node.js functions allow reusable, modular code and are essential for handling asynchronous tasks efficiently. Using callbacks, Promises, or async/await, developers can manage operations like file handling, network requests, and database interactions ~~effici~~ effectively.



Practical No.:- 04

Aim: using File handling demonstrate all basic file operations (create, write, Read and Delete)

Theory:

Node.js Provides the functionality to read and write files from the computer. Reading and Writing the file in Node.js is done by using Node.js modules called fs module, it is one of the most well-known built-in Node.js modules out there.

Common use for the File system module:

- Read, create, update and delete files.

1. Creating and writing to a file -

- You can use `fs.writeFile()` to create a new file or overwrite an existing file. If the file doesn't exist, it will be created.

2. Reading from a file -

- To read the content of a file, you can use `fs.readFile()`.

3. Deleting a file -

- You can use `fs.unlink()` to delete a file.



Conclusion:

- In this practical, we successfully demonstrated basic file handling operations in Node.js using the file system (fs) module. The experiment covered creating, writing, reading, and deleting files - all of which are essential for handling data storage and manipulation in backend development. By using methods like `fs.writeFile()`, `fs.readFile()`, and `fs.unlink()`, we learned how Node.js interacts with the file system asynchronously, ensuring efficient and non-blocking I/O operations. This practical enhanced our understanding of managing files dynamically in real-world Node.js applications.



Practical No.:- 05

Aim: create an HTTP server and Perform operations on it.

Theory:

In Node.js the HTTP module is used to create and manage web servers. It enables the server to listen for client requests and send responses accordingly. Node.js follows an event-driven, non-blocking I/O model, which makes it efficient for handling multiple client connections simultaneously.

- To build an HTTP server, we use the `http.createServer()` method. This method accepts a callback function with two parameters:
 - `req(request)`: contains details about the client's request (URL, method, headers, etc.).
 - `res(response)`: is used to send data or messages back to the client.
- After creating the server, it must be assigned to a specific port using the `server.listen()` method (e.g. Port 3000). Once started, it continuously listens for and processes incoming requests.



Conclusion:

- In this Practical, we learned how to create and manage an HTTP server in Node.js using the built-in `http` and `fs` modules. The server was designed to handle multiple routes and perform basic file operations such as reading and writing files through HTTP requests. This experiment demonstrated how Node.js efficiently handles client requests and serves response asynchronously, making it ideal for backend and web server development. Overall, the Practical provided a clear understanding of how HTTP servers work in Node.js and how they can be used to perform CRUD-like operations dynamically.